

DAY 2 · FastAPI learning notes

Path & Query Parameter Validation

Path parameters and numeric validations · Query parameters and string validations

Learning objectives

After working through these notes you should be able to:

- ✓ Tell a path parameter apart from a query parameter by looking at the URL and the function signature.
- ✓ Add numeric rules to a path parameter with `Path(gt=..., le=...)`.
- ✓ Add string rules to an optional query parameter with `Query(min_length=..., max_length=...)`.
- ✓ Explain why invalid input never reaches your function body, and read the 422 error FastAPI sends back.

1. The big idea: let FastAPI guard the door

Every request carries data in its URL. Before your function runs, FastAPI reads that data, converts it to the Python types you declared, and checks any rules you attached. If something is wrong, FastAPI answers the client itself with **422 Unprocessable Entity** and your code never runs.

That means your function body can assume the data is already clean. You declare the rules once, in the signature, and you also get them documented automatically in `/docs`.

Where the two kinds of parameter live in a URL

```
GET /items/5?q=hello
      ^^^^^^^
      | --- query parameter: q = "hello"    (after the ?)
      +---- path parameter: item_id = 5     (part of the path)
```

2. Path parameters and numeric validations

What it is: `Path()` attaches validation and metadata to a path parameter, the same way `Query()` does for query parameters.

Rule	Meaning	Example: value 0 with this rule
<code>gt</code>	greater than	<code>gt=0</code> → rejected
<code>ge</code>	greater than or equal	<code>ge=0</code> → accepted
<code>lt</code>	less than	<code>lt=1000</code> → accepted
<code>le</code>	less than or equal	<code>le=1000</code> → accepted

Why it matters: a path parameter is always required, because it is part of the URL itself. So `Path()` is never used to make something optional. It exists purely to add rules and better docs, e.g. rejecting `item_id=0` before your function body ever runs.

Remember

The `g` / `l` stands for greater / less, the `t` for than, the `e` for or equal. `gt=0` excludes 0; `ge=0` includes it.

3. Query parameters and string validations

What it is: any function parameter that is not in the path string becomes a query parameter. `Query()` adds string rules to it: `min_length`, `max_length`, and `pattern` (a regular expression).

A query parameter becomes **optional** when you give it a default value, usually `None`. The length rules are only checked when the client actually sends the parameter.

Extra detail worth knowing: alias

`alias=` lets the parameter have a URL name that is not a valid Python identifier (for example `item-query`), while your function still uses a clean Python variable name internally.

alias in one line (from the Day 2 summary; not used in main.py)

```
q: Annotated[str | None, Query(alias="item-query")] = None
# URL: /items/?item-query=foo -> q == "foo"
```

4. Why Annotated?

`Annotated[int, Path(gt=0)]` reads as “an `int`, plus some extra information”. Python only cares about the first part (the type). FastAPI also reads the extra part (the rules). Keeping the rules inside `Annotated` leaves the `= None` slot free for the real default value, which is the modern recommended style.

5. Walkthrough of your code (day2/main.py)

```
@app.get("/items/{item_id}")
async def get_item_id(item_id: Annotated[int, Path(gt=0, le=1000)]
                      , q: Annotated[str | None, Query(min_length=3, max_length=50)] = None ):
    return {"id": item_id,
            "q": q}
```

- 1 `@app.get("/items/{item_id}")` registers the function for GET requests. `{item_id}` marks a path parameter.
- 2 `item_id` matches the name in braces, so it is read from the path. `int` converts the text `"5"` to `5`. `Path(gt=0, le=1000)` allows 1 to 1000 only.
- 3 `q` is not in the path, so it is a query parameter. `str | None` plus `= None` makes it optional. `Query(min_length=3, max_length=50)` limits its length when it is sent.
- 4 Only if every check passes does FastAPI call the function. The returned dict is converted to JSON.

In your code

The TODO suggested returning the key `"item_id"`; your solution returns `"id"`. Both are correct – it only changes the key name in the JSON response.

6. The four test cases (verified)

These are the cases from the exercise, run against your code:

Request	Result	Why
<code>/items/5</code>	200 <code>{"id": 5, "q": null}</code>	valid id, q omitted → default None
<code>/items/0</code>	422	violates <code>gt=0</code>
<code>/items/5?q=hi</code>	422	"hi" has 2 characters, violates <code>min_length=3</code>

Request	Result	Why
/items/5?q=hello	200 {"id": 5, "q": "hello"}	passes both checks
/items/abc (extra)	422	"abc" cannot be converted to <code>int</code>

Reading a 422 error

Actual response for /items/5?q=hi

```
{
  "detail": [
    {
      "type": "string_too_short",
      "loc": ["query", "q"],
      "msg": "String should have at least 3 characters",
      "input": "hi",
      "ctx": {"min_length": 3}
    }
  ]
}
```

`loc` says **where** the problem is (`query` → `q`), `msg` says **what** is wrong, `ctx` names the rule. For `/items/0` you get `type: "greater_than"` with `loc: ["path", "item_id"]`.

The prediction questions, answered

- **Does the function body run for /items/0?** No. FastAPI validates first and returns 422 before calling it. That is good API design: invalid input should never reach your business logic.
- **Does the error for q=hi name the field and rule?** Yes: `loc` names the field, `msg` and `ctx` name the rule.

7. Path vs query at a glance

	Path parameter	Query parameter
Where in the URL	inside the path: <code>/items/5</code>	after <code>?:</code> <code>?q=hello</code>
How FastAPI decides	name appears in <code>{...}</code> in the route	any other simple-typed parameter
Required?	always	only if it has no default
Validation helper	<code>Path()</code>	<code>Query()</code>
Typical rules	numeric: <code>gt</code> , <code>ge</code> , <code>lt</code> , <code>le</code>	string: <code>min_length</code> , <code>max_length</code> , <code>pattern</code>

8. Common mistakes

- Trying to make a path parameter optional with `= None` – path parameters are always required.
- Mixing up `gt` and `ge` (is 0 allowed or not?).
- Forgetting the default `= None`, which silently makes a query parameter required.
- Checking the rules again by hand inside the function – FastAPI has already done it.
- Expecting `min_length` to make `q` required. It only applies when `q` is sent.

9. Revision summary

- Path parameters live in the path and are always required; `Path()` adds numeric rules like `gt` and `le`.
- Query parameters come after `?`; a default value (usually `None`) makes them optional; `Query()` adds string rules.

- Write rules with `Annotated[type, Path(...)] / Annotated[type, Query(...)]`.
- Invalid input → FastAPI returns 422 with `loc`, `msg` and `ctx`; your function never runs.
- Rules show up automatically in `/docs` – test every case there.